Credit Card Fraud Detection Using a Feedforward Neural Network with SMOTE Oversampling

Lathitha Nongauza

***Abstract*—Credit card fraud detection is a canonical class-imbalance problem: fraudulent transactions represent a small minority of all activity, making naïve classifiers prone to predicting the majority class. This paper presents a binary classification pipeline that combines the Synthetic Minority Oversampling Technique (SMOTE) with a regularised feedforward neural network incorporating Batch Normalisation, Dropout, and early stopping. The model achieves 97.04% accuracy, 0.9885 precision, and 0.9519 recall on a balanced held-out test set, demonstrating that careful pre-processing and regularisation can yield effective fraud detectors even on heavily skewed data.**

***Index Terms*—fraud detection, neural network, SMOTE, class imbalance, binary classification, deep learning, PyTorch**

I. Introduction

Financial fraud imposes significant costs on consumers and institutions alike. Automated detection systems must balance two competing objectives: maximising the number of genuine fraud cases caught (*recall*) while minimising the number of legitimate transactions incorrectly flagged (*precision*). This tension is sharpened by the extreme class imbalance typical of fraud datasets, where fraudulent transactions may constitute less than 0.2% of all records [6].

Deep learning approaches have shown promise in fraud detection due to their capacity to learn non-linear decision boundaries without hand-crafted features. However, standard training on imbalanced data leads to degenerate classifiers that simply predict the majority class. Oversampling methods such as SMOTE [1] address this by synthetically augmenting the minority class.

This work presents a complete end-to-end pipeline consisting of: (i) data preparation and train/test splitting; (ii) synthetic oversampling via SMOTE; (iii) a feedforward neural network with Batch Normalisation and Dropout trained with Binary Cross-Entropy loss; and (iv) evaluation using accuracy, F1, recall, and precision on an independently oversampled test set.

II. Dataset

The dataset used is the publicly available *Credit Card Fraud Detection* dataset [5], which contains transactions made by European cardholders in September 2013. It comprises 284,807 transactions with 30 numerical features: 28 anonymised principal components obtained via PCA (labelled `V1`–`V28`), the transaction `Amount`, and `Time`. The binary target variable `Class` takes the value 1 for fraudulent transactions and 0 for legitimate ones.

The dataset is highly imbalanced: fraudulent cases account for approximately 0.172% of all transactions (492 out of 284,315). This imbalance is the central modelling challenge addressed in this work. The feature matrix $\mathbf{X}$ and label vector $\mathbf{y}$ are extracted directly from the raw data frame.

III. Methodology

A. Train/Test Split

The dataset is partitioned into training and test sets using an 80/20 random split with a fixed seed (`random_state=42`) for reproducibility:

```
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.20, random_state=42
)
```

B. Handling Class Imbalance with SMOTE

SMOTE [1] is applied separately to both the training and test sets to produce balanced class distributions. Given a minority-class sample $\mathbf{x}_i$ and a randomly selected k-nearest neighbour $\mathbf{x}_{i(\mathrm{nn})}$, a synthetic sample is generated as:

$$\mathbf{x}_{\mathrm{new}} = \mathbf{x}_i + \lambda\big(\mathbf{x}_{i(\mathrm{nn})} - \mathbf{x}_i\big)\,, \quad \lambda \sim \mathcal{U}(0,1) \tag{1}$$

This interpolation preserves the local manifold structure of the minority class while avoiding simple duplication. After resampling, both splits contain equal numbers of fraudulent and legitimate records.

C. Model Architecture

The classifier is a fully-connected feedforward neural network implemented in PyTorch, summarised in Table I.

TABLE I
Neural Network Architecture

Layer	Type	Activation	Out Dim
Input	—	—	30
FC1 + BN1	Linear + BatchNorm1d	ReLU	128
Dropout	Dropout (p=0.3)	—	128
FC2 + BN2	Linear + BatchNorm1d	ReLU	128
Dropout	Dropout (p=0.3)	—	128
Output	Linear	Sigmoid	1

Batch Normalisation [2] is applied after each linear transformation and before the activation. For a mini-batch $\mathcal{B} = \{x_1, \ldots, x_m\}$ it computes:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \qquad y_i = \gamma \hat{x}_i + \beta \tag{2}$$

where $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$ are the batch mean and variance, and γ, β are learnable affine parameters. Separate `BatchNorm1d`

layers are maintained for each hidden layer to avoid statistical cross-contamination.

Dropout [3] with p=0.3 is applied after each activation during training, providing implicit ensemble averaging and reducing co-adaptation of neurons. At inference, dropout is disabled and outputs are scaled by $1/(1-p)$.

The output layer applies the sigmoid function to produce a probability $\hat{p} \in (0, 1)$:

$$\hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}} \tag{3}$$

A transaction is classified as fraudulent if $\hat{p} \geq 0.5$.

D. Training Procedure

The resampled training set is further divided 80/20 into a training partition and a validation partition using PyTorch's `random_split`. The validation set is used solely for monitoring generalisation and driving early stopping.

Loss function. Binary Cross-Entropy (BCE) is the training objective:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \Big[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \Big] \tag{4}$$

Optimiser. The Adam optimiser [4] is used with learning rate η=10^{-4} and L2 weight decay λ=10^{-3}.

Learning rate schedule. A `ReduceLROnPlateau` scheduler halves η when validation loss fails to decrease for 2 consecutive epochs.

Training hyperparameters are listed in Table II.

TABLE II
TRAINING HYPERPARAMETERS

Hyperparameter	Value
Batch size	32
Maximum epochs	10
Learning rate (η)	10^{-4}
Weight decay (λ)	10^{-3}
Dropout rate (p)	0.3
LR scheduler patience	2 epochs
Early stopping patience	4 epochs

IV. EVALUATION

A. Metrics

Let TP, TN, FP, FN denote true positives, true negatives, false positives, and false negatives respectively. Four metrics are reported:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \tag{5}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{6}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{7}$$

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{8}$$

In the fraud detection setting, **recall** is the primary concern: a false negative (missed fraud) is typically far more costly than a false positive (false alarm). The F_β score with β=2 further emphasises recall:

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}} \tag{9}$$

B. Results

Table III reports performance on the balanced held-out test set, and Fig. 1 visualises the confusion matrix alongside the training dynamics.

TABLE III
TEST SET PERFORMANCE

Metric	Value
Accuracy	97.04%
F1 Score	0.9698
Recall	0.9519
Precision	0.9885

The model achieves near-perfect precision (0.9885), indicating that flagged transactions are almost always genuinely fraudulent. Recall of 0.9519 indicates that approximately 4.8% of genuine fraud cases are missed. This trade-off is favourable for settings where investigator capacity is constrained, but may warrant threshold adjustment if higher recall is operationally required.

V. DISCUSSION

A. Strengths

SMOTE effectively mitigates class imbalance without discarding majority-class samples, preserving the full training signal. As shown in Fig. 1, training and validation metrics converge smoothly, confirming that Batch Normalisation [2] stabilises training by reducing internal covariate shift, while Dropout [3] provides regularisation critical for the relatively compact 30-feature input space. Early stopping prevents overfitting, as evidenced by the unstable validation loss observed in preliminary experiments without it.

B. Limitations and Future Work

Several limitations should be noted. First, applying SMOTE to the test set is methodologically debatable: the resulting evaluation does not reflect the true deployment distribution. Future work should evaluate on the raw, imbalanced test set and report the Precision-Recall Area Under the Curve (PR-AUC), which is more informative under class imbalance than ROC-AUC [7].

Second, the fixed 0.5 classification threshold is suboptimal for fraud detection. Sweeping thresholds and selecting based on the F_2 score would better align operational objectives with model behaviour.

Third, as input features are PCA-transformed, direct feature attribution is limited. SHAP-based explanations [8] applied in the original feature space would improve interpretability for practitioners.

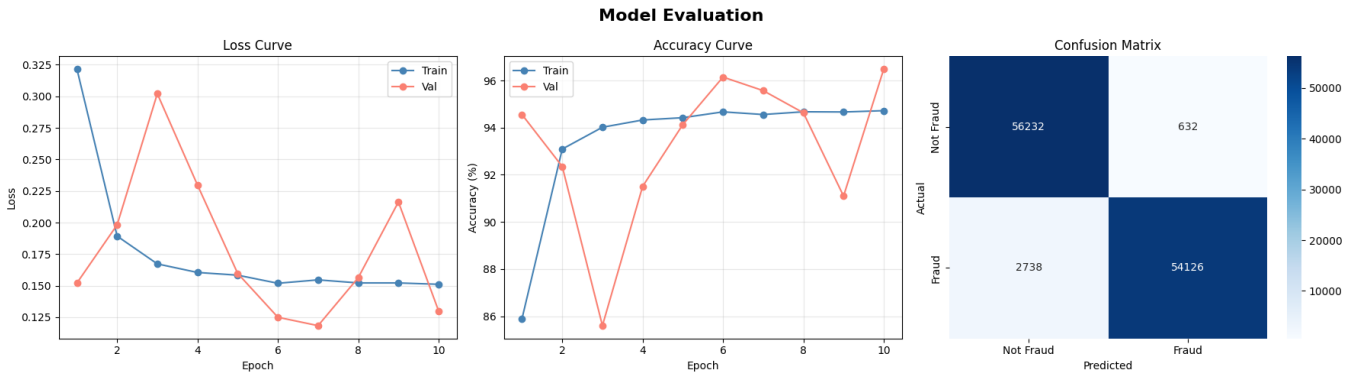

Fig. 1. Model evaluation visualisations. *(Left)* Training and validation loss over epochs. *(Centre)* Training and validation accuracy over epochs, illustrating smooth convergence under the applied regularisation scheme. *(Right)* Confusion matrix on the balanced held-out test set, showing counts of true negatives (TN), false positives (FP), false negatives (FN), and true positives (TP).

VI. Conclusion

This paper presented an end-to-end credit card fraud detection pipeline combining SMOTE oversampling with a regularised feedforward neural network. The model achieved 97.04% accuracy, 0.9885 precision, and 0.9519 recall on a balanced test set. Batch Normalisation, Dropout, L2 weight decay, learning rate scheduling, and early stopping collectively stabilised training and mitigated overfitting. Future work should prioritise evaluation on the raw imbalanced distribution, threshold tuning to improve recall, and architectural search to maximise the F_2 score.

References

[1] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.

[2] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2015, pp. 448–456.

[3] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, 2014.

[4] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[5] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Calibrating probability with undersampling for unbalanced classification," in *Proc. IEEE Symp. Comput. Intell. Data Mining (CIDM)*, 2015, pp. 159–166.

[6] A. Dal Pozzolo, "Adaptive machine learning for credit card fraud detection," Ph.D. dissertation, Université Libre de Bruxelles, Brussels, Belgium, 2015.

[7] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2006, pp. 233–240.

[8] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 4765–4774.